What we're doing

exp82 is a contact-prediction inference-algorithm harness for the contacts-v1 LM. The question: given a protein's sequence, which decoder extracts the most residue–residue contact signal — `pairwise` scoring, `rollout` frequency-voting, or exp27-style `iterative` growing-K refinement? This PR re-runs that search on our **best** model — the #61/#75 tuned 1.5B (eval loss 2.7566) that #89 exported — and picks + costs the best inference recipe.

The original finding (#67 quick model, eval loss 2.98)

All three methods rank long-range contacts ~2× random but at ~1–2% absolute precision; `iterative` was marginally best, `rollout` no better than `pairwise`. Verdict: "better inference doesn't rescue a weak base model — the bottleneck is the base model, not the readout." The open follow-up was: re-run on the tuned model.

The open question

#89 took the tuned model (long-range ranking AUC 0.62 → 0.88) but scored **pairwise only**, assuming exp82's "smarter decoders don't help." That came from the *weak* model — yet exp27 got **+30% from iteration on a strong base model**. So: now that the base model is strong, does the inference algorithm matter again?

Method selection on a dev set (no test-set hill-climbing)

We *select* the algorithm on a 16-protein **FoldBench dev** set (L 100–250, fixed seed-0 sample). Held out for final evaluation: the other 84 FoldBench + 454 denovo/CASP/CAMEO proteins + the entire contacts-v1 test split. Candidate universe = resolved residues; GT = pyconfind contacts (sep≥6), identical to #89.

Dev result: rollout > pairwise > iterative (a flip)

long-range R-precision: pairwise 0.16 · **rollout 0.20** · iterative 0.13 · random 0.01.
On the strong model the order inverts vs #67: `rollout` beats `pairwise` on every metric (+20% R), and `iterative` now **hurts** (−18% R). iterative commits the model's own top-K (at ~13–33% precision) as fixed context, seeding mostly *false* contacts that the co-occurrence prior then propagates. `rollout` is variance reduction; its gain is largest at the very top of the ranking.

Sampling sweep: a wash

Sweeping temperature (1.0 / 0.7 / 0.5), top-p, and a domain-aware top-k = L/5 moves R-precision by < ±0.006 — within noise. T=0.5 clearly hurts (over-sharpening collapses the vote). Default T=1.0 is near-optimal: the lever is the *method*, not the sampling distribution's shape.

Resampling the document per rollout: a small, free bonus

Give each rollout a fresh contacts-v1 realization (resample the N-terminus start + statement order, #89-style TTA) → the vote averages over the document nuisance as well as the sampling noise, at ~no extra GPU cost. +0.007 R on dev, consistent across all 7 metrics. Smaller than #89's +0.05 pairwise TTA — rollout already ensembles. Settled recipe: **rollout + resample**.

Full curated eval (554 proteins): vs every predictor

Scored by #89's exact metric code, so comparable to the structure predictors. Long-range R-precision: rollout+resample **0.355** > K=10 pairwise ensemble 0.315 > pairwise 0.269 — the **best LM-only inference**, likewise on contacts@L (0.232 / 0.209 / 0.188). It trails every structure predictor on top-K (ESMFold2 0.769, ESMFold 0.732, Protenix-MSA 0.628). Plots: `plots/cmp_*_by_config_and_range.png` (+ by MSA-depth tier / fold novelty).

Tie-break (raised in review): recover the AUC for free

Vote counts are integers, so ~⅔ of candidate pairs tie at 0 votes — that arbitrarily-ordered tail drags AUC below pairwise's (0.851 vs 0.881) despite better top-K. Break ties with the pairwise log-prob (`votes + min-max(pairwise)∈[0,0.5)`, votes stay primary): AUC **0.851 → 0.898** (level with the ensemble, above ESMFold's 0.892) at **zero top-K cost**. Free — the pairwise matrices already exist. Headline recipe: **rollout+resample +tiebreak**.

Cost

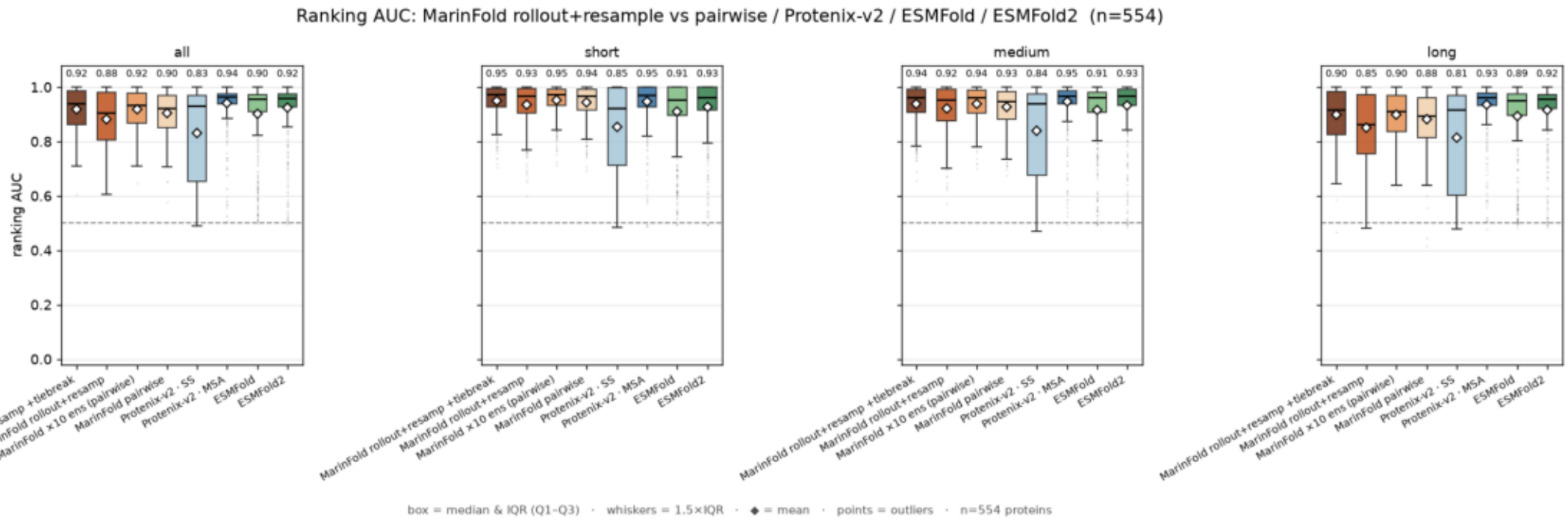
rollout+resample: **~50 s/protein** mean on one A5000 at n=100 (median 44 s, max 225 s at L=738), ~linear in L; the score-matrix re-run (adaptive batch ≤64) did 554 in **3.9 h**. pairwise is ~0.3 s, and the tie-break is **free**. Rollouts terminate cleanly — **0% hit the 4·L+64 cap** — emitting ~2–2.5·L tokens / ~80–125 contacts each. So the top-K gain costs ~150× pairwise per protein: cheap in absolute terms (no retraining), not free at scale.

Conclusion

Careful tuning flipped exp82's verdict. On the strong model, **better inference does help**: rollout + per-rollout document-resampling + pairwise tie-breaking is the **best LM-only inference** — long-range R-precision 0.355 / contacts@L 0.231 (vs pairwise 0.269 / 0.188) with AUC 0.898, all free (no retraining). `iterative` still hurts (self-seeding needs higher base precision than this model has). The decoder is a real lever again — but closing the gap to the structure predictors (ESMFold2 0.769 R) still needs a stronger model, not just a better readout.

cmp_contacts_at_AUC_by_config_and_range.png

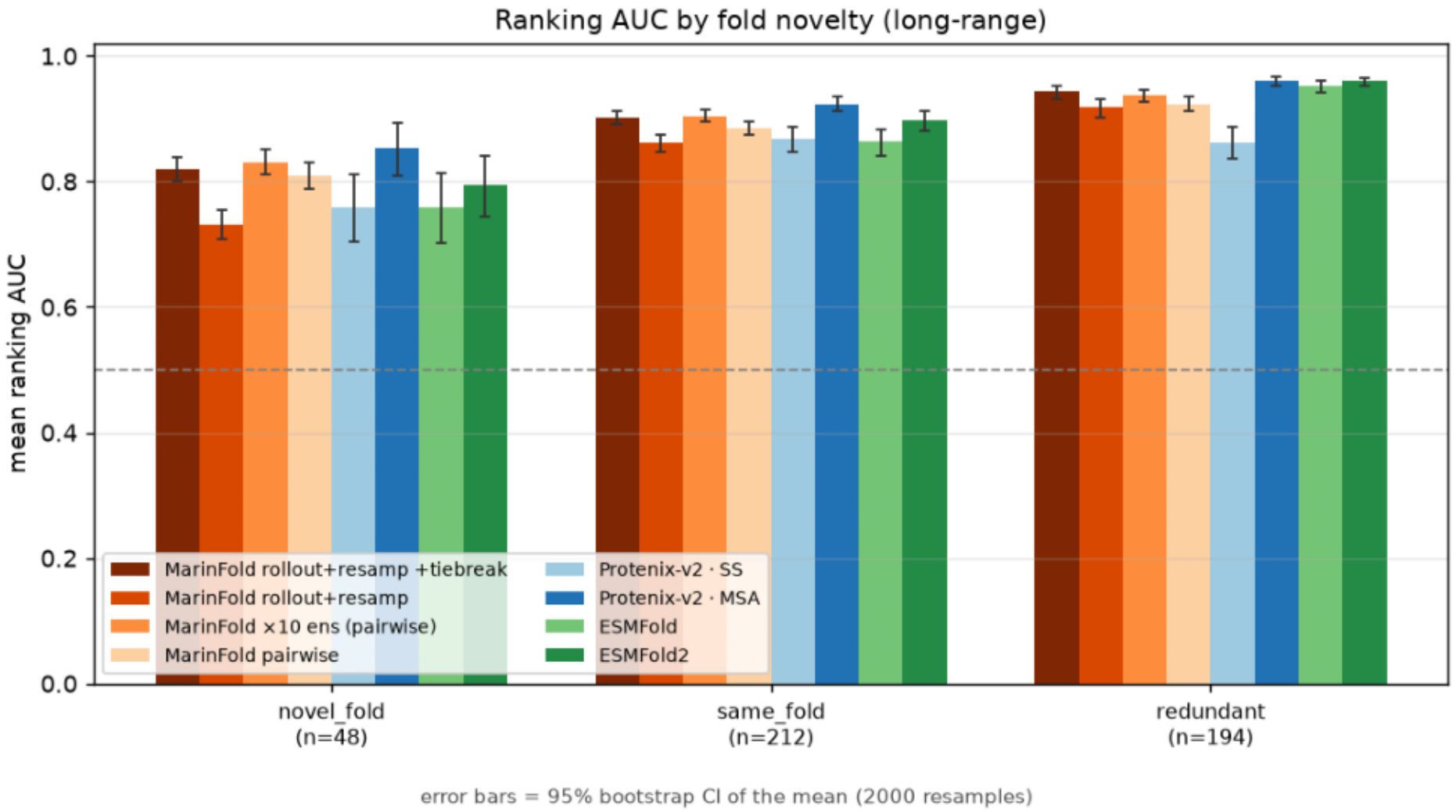
ranking AUC vs pyconfind contacts (degree>=0.001, sep>=6), per predictor, aggregate and by range, over 554 proteins. Boxplots: box = median & IQR, whiskers = 1.5×IQR, points = outliers, white diamond = mean (labelled). rollout+resample ranks pairs by sampled-completion vote frequency; pairwise by contact-statement log-prob; structure models by pyconfind degree on the predicted structure. Metrics computed by exp89's exact code.




```
$ plot_comparison.py --precision-csv _scratch/contact_precision_with_rollout.csv --out plots
```

cmp_contacts_at_AUC_by_fold_verdict.png

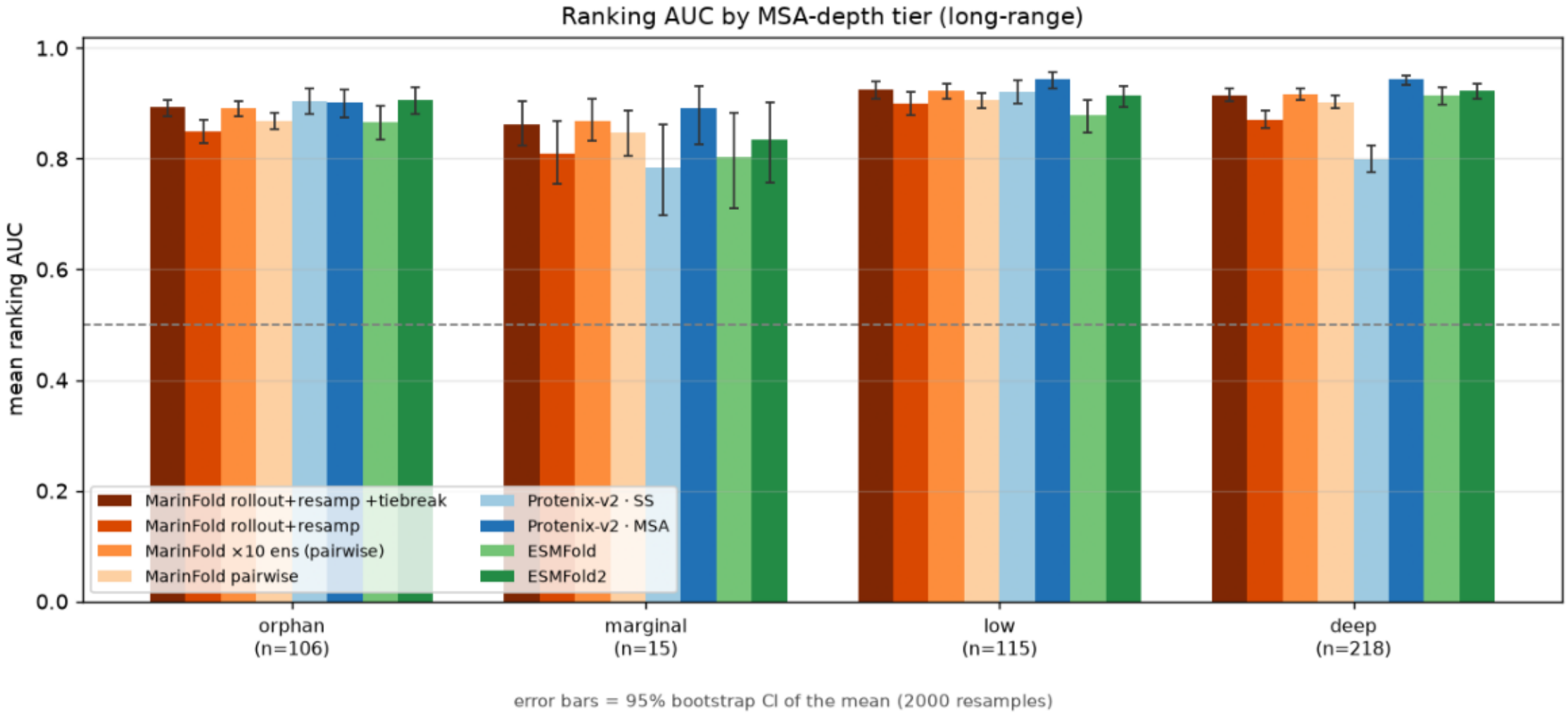
Ranking AUC by fold novelty (long-range). Bars = mean; error bars = 95% bootstrap CI of the mean.




```
$ plot_comparison.py --precision-csv _scratch/contact_precision_with_rollout.csv --out plots
```

cmp_contacts_at_AUC_by_neff_tier.png

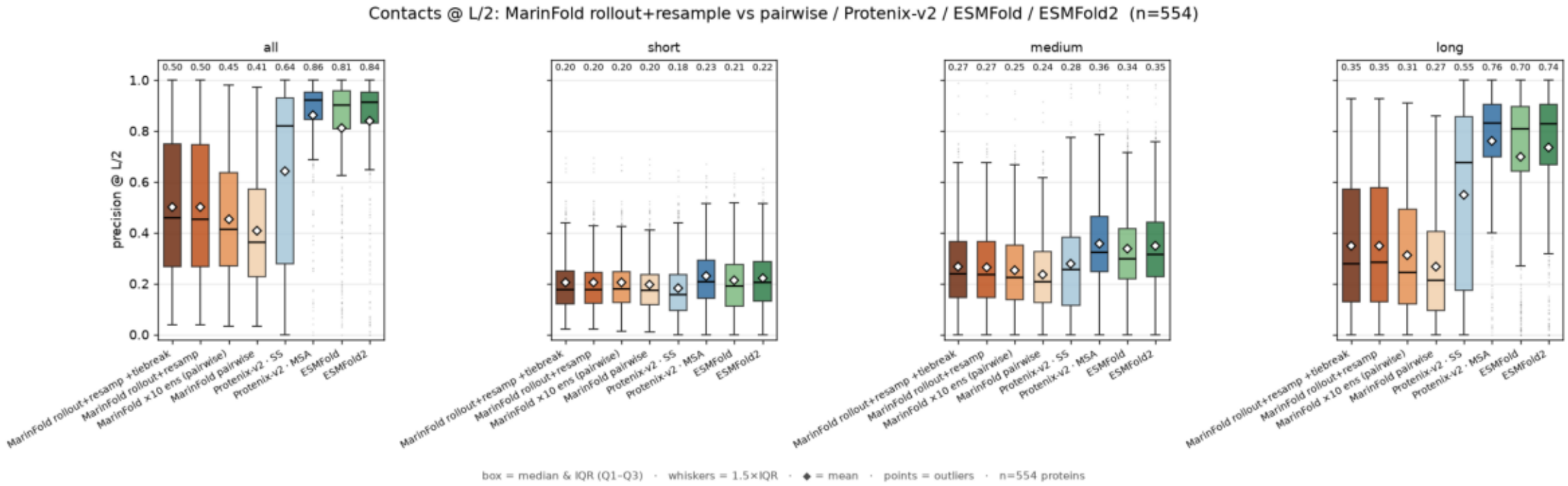
Ranking AUC by MSA-depth tier (long-range). Bars = mean; error bars = 95% bootstrap CI of the mean.




```
$ plot_comparison.py --precision-csv _scratch/contact_precision_with_rollout.csv --out plots
```

cmp_contacts_at_L_2_by_config_and_range.png

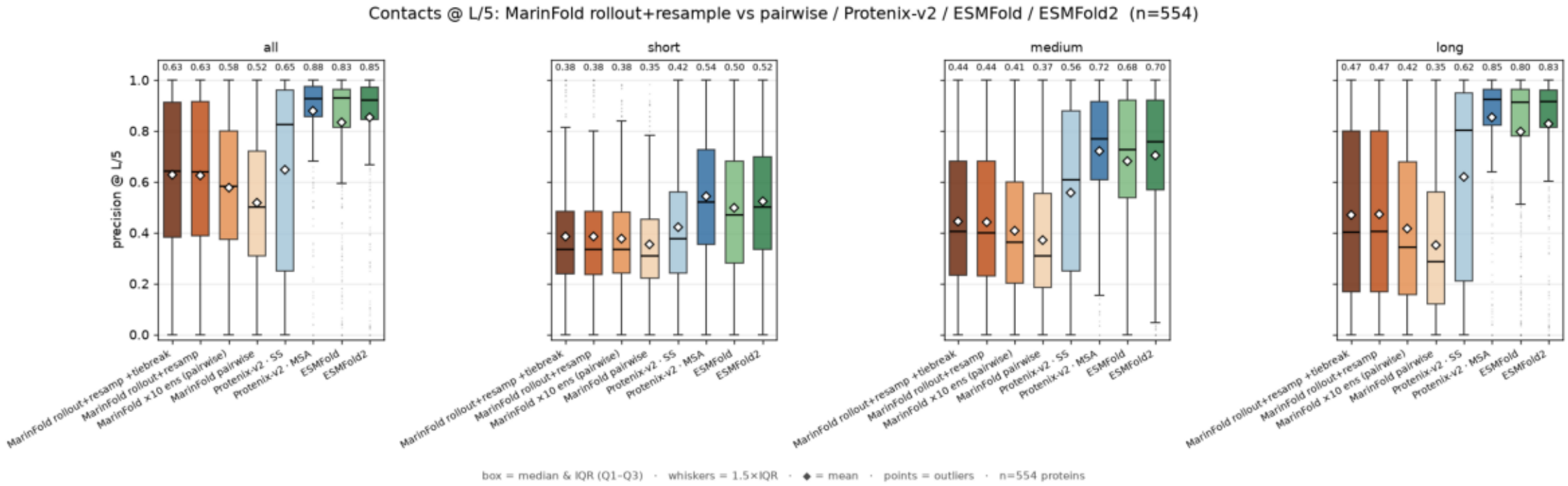
precision @ L/2 vs pyconfind contacts (degree>=0.001, sep>=6), per predictor, aggregate and by range, over 554 proteins. Boxplots: box = median & IQR, whiskers = 1.5×IQR, points = outliers, white diamond = mean (labelled). rollout+resample ranks pairs by sampled-completion vote frequency; pairwise by contact-statement log-prob; structure models by pyconfind degree on the predicted structure. Metrics computed by exp89's exact code.




```
$ plot_comparison.py --precision-csv _scratch/contact_precision_with_rollout.csv --out plots
```

cmp_contacts_at_L_5_by_config_and_range.png

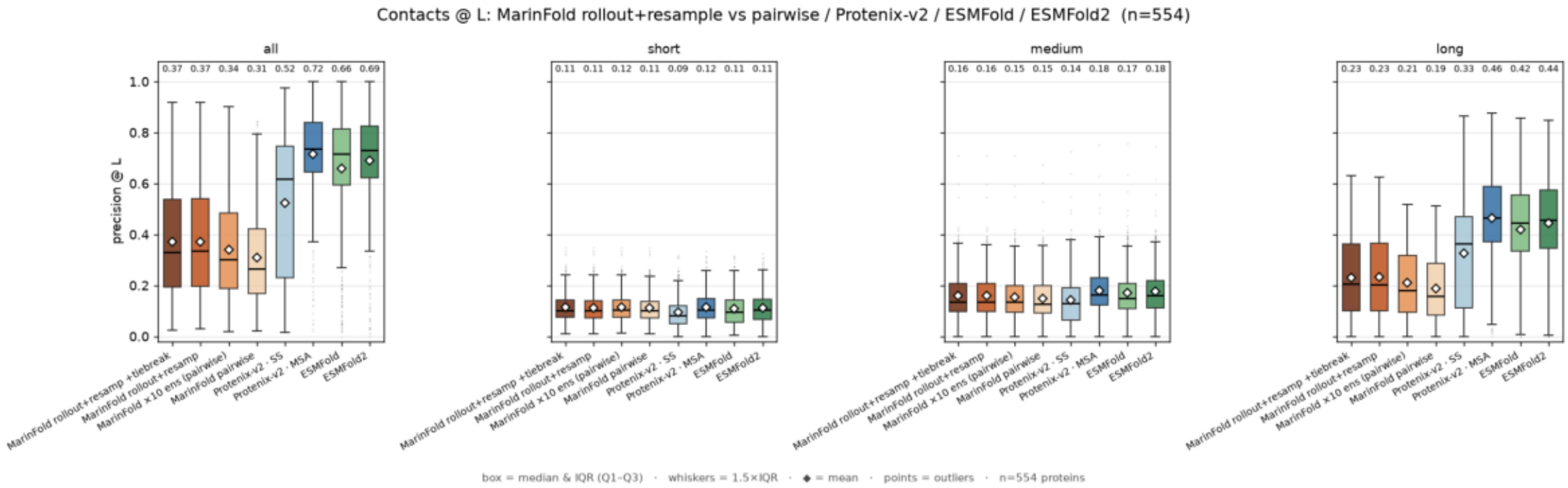
precision @ L/5 vs pyconfind contacts (degree>=0.001, sep>=6), per predictor, aggregate and by range, over 554 proteins. Boxplots: box = median & IQR, whiskers = 1.5×IQR, points = outliers, white diamond = mean (labelled). rollout+resample ranks pairs by sampled-completion vote frequency; pairwise by contact-statement log-prob; structure models by pyconfind degree on the predicted structure. Metrics computed by exp89's exact code.




```
$ plot_comparison.py --precision-csv _scratch/contact_precision_with_rollout.csv --out plots
```

cmp_contacts_at_L_by_config_and_range.png

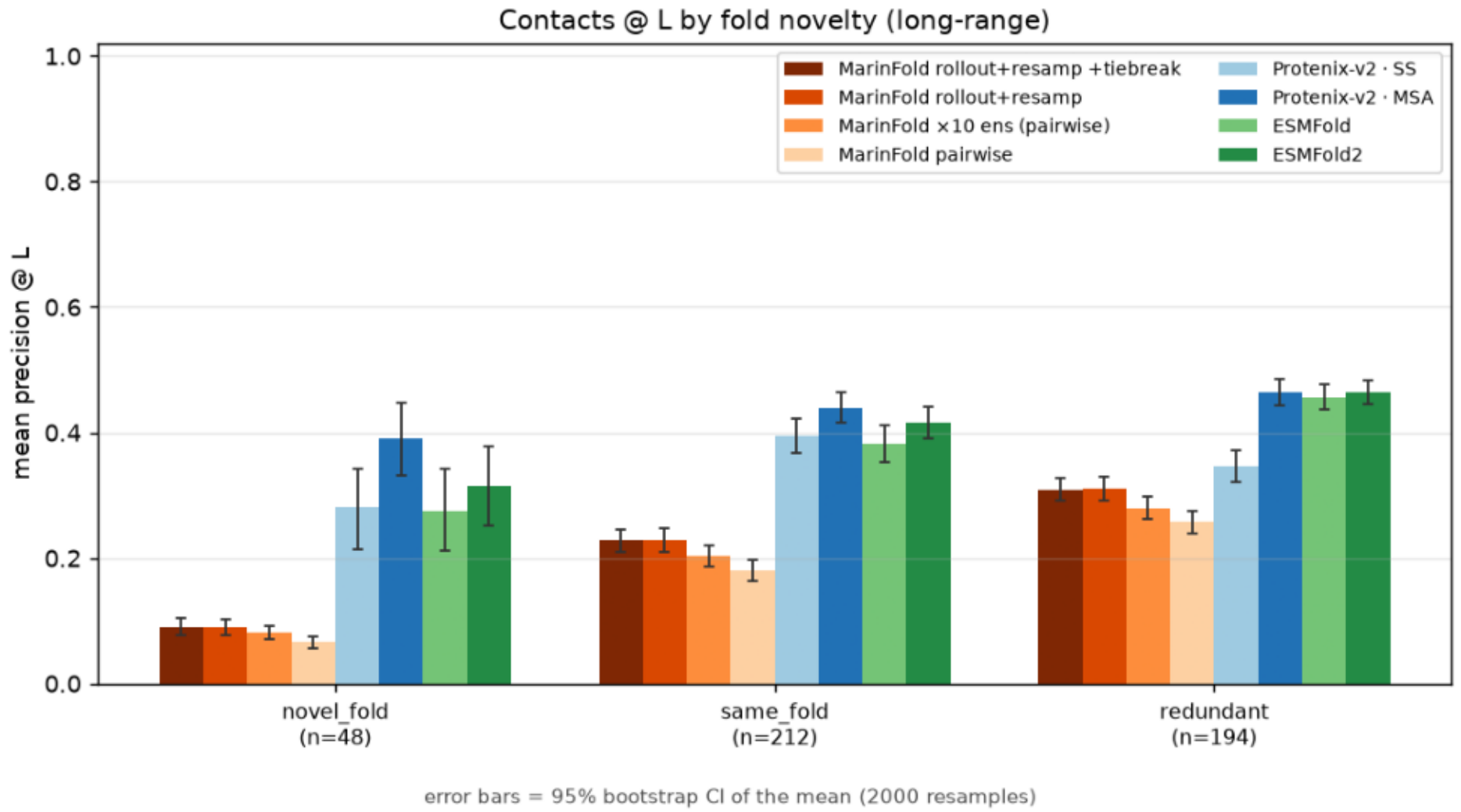
precision @ L vs pyconfind contacts (degree>=0.001, sep>=6), per predictor, aggregate and by range, over 554 proteins. Boxplots: box = median & IQR, whiskers = 1.5×IQR, points = outliers, white diamond = mean (labelled). rollout+resample ranks pairs by sampled-completion vote frequency; pairwise by contact-statement log-prob; structure models by pyconfind degree on the predicted structure. Metrics computed by exp89's exact code.




```
$ plot_comparison.py --precision-csv _scratch/contact_precision_with_rollout.csv --out plots
```

cmp_contacts_at_L_by_fold_verdict.png

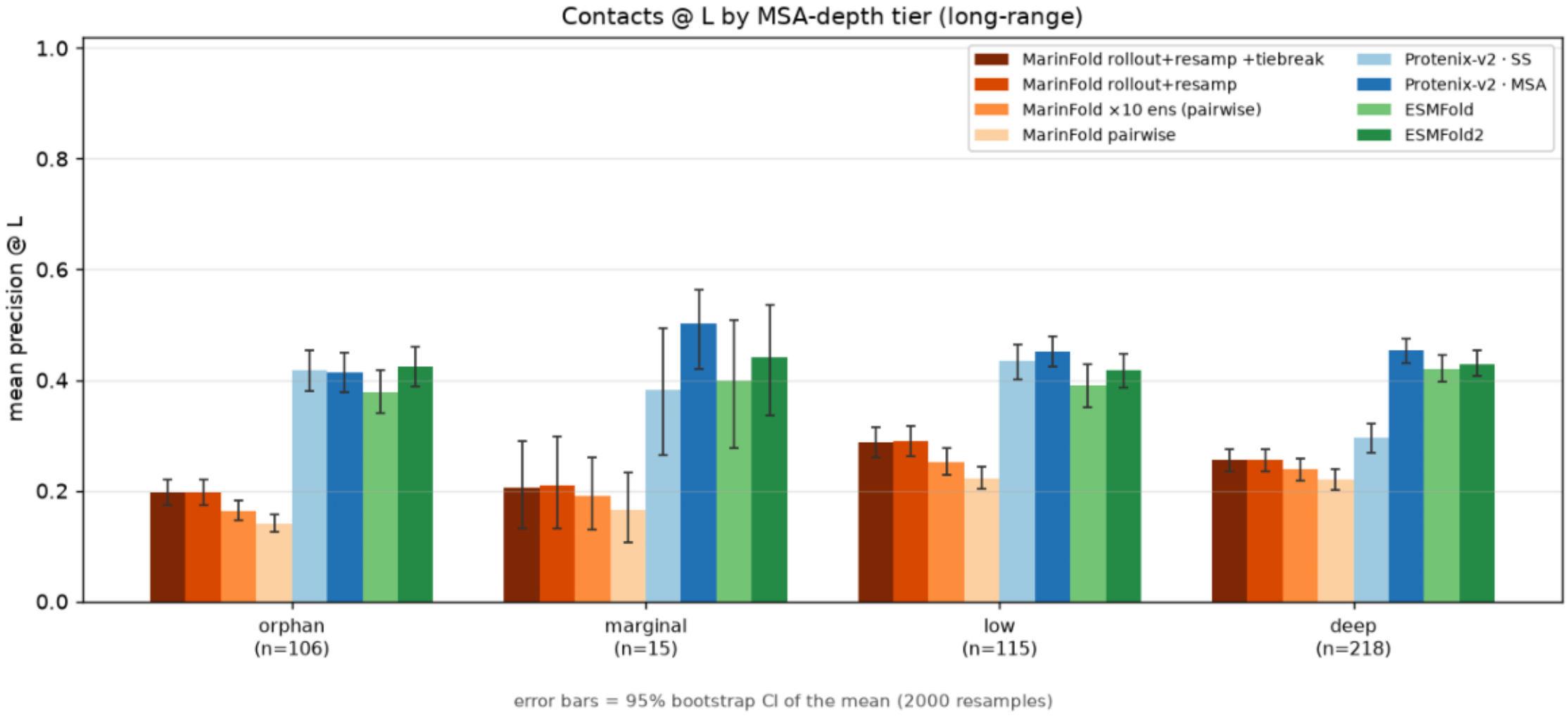
Contacts @ L by fold novelty (long-range). Bars = mean; error bars = 95% bootstrap CI of the mean.




```
$ plot_comparison.py --precision-csv _scratch/contact_precision_with_rollout.csv --out plots
```

cmp_contacts_at_L_by_neff_tier.png

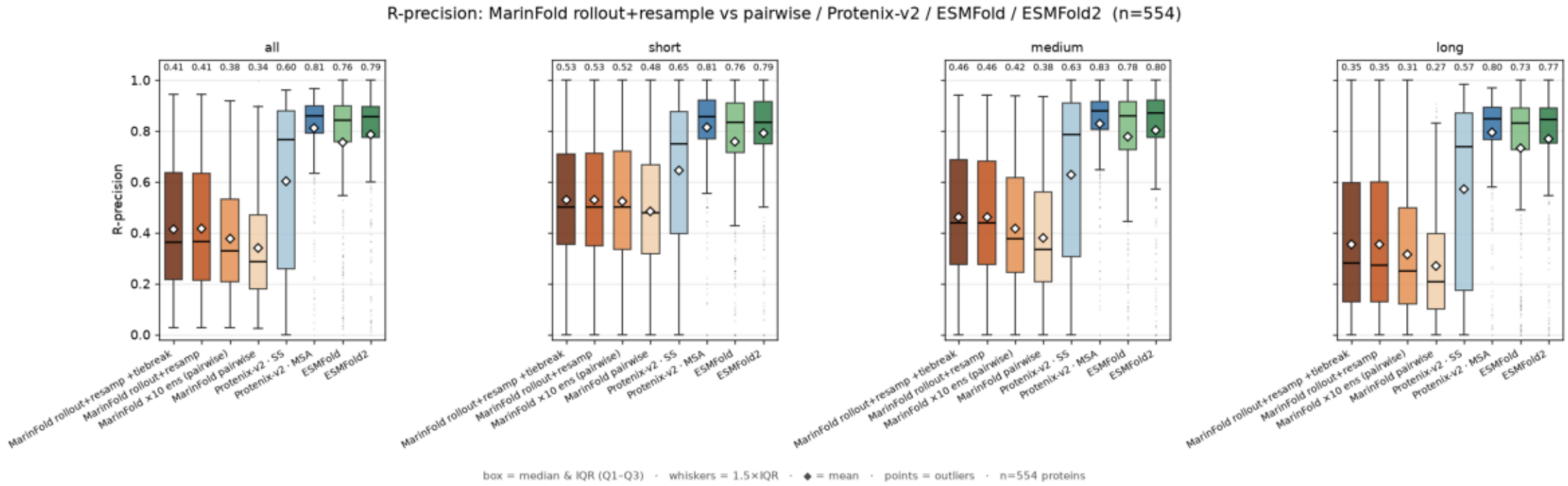
Contacts @ L by MSA-depth tier (long-range). Bars = mean; error bars = 95% bootstrap CI of the mean.




```
$ plot_comparison.py --precision-csv _scratch/contact_precision_with_rollout.csv --out plots
```

cmp_contacts_at_R_by_config_and_range.png

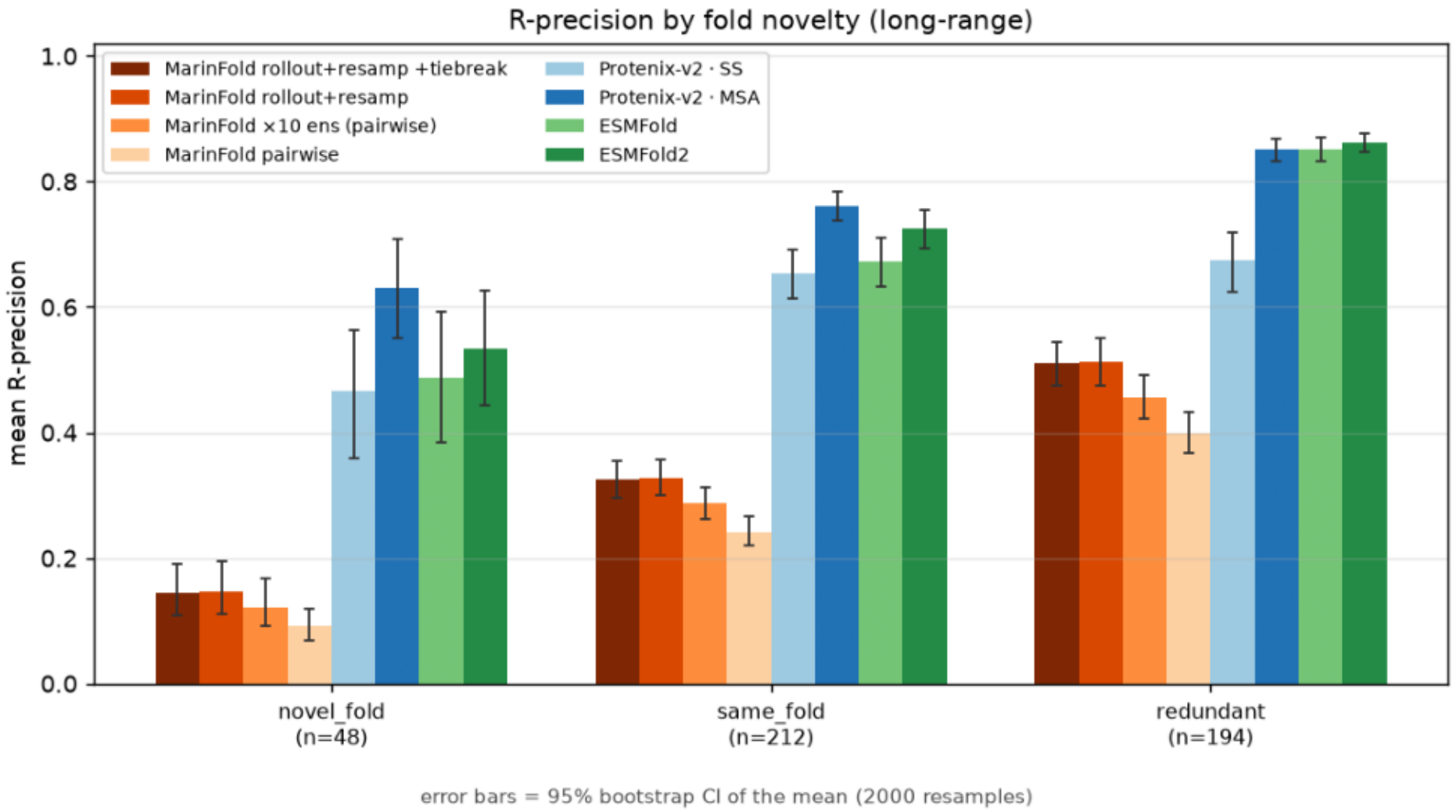
R-precision vs pyconfind contacts (degree>=0.001, sep>=6), per predictor, aggregate and by range, over 554 proteins. Boxplots: box = median & IQR, whiskers = 1.5×IQR, points = outliers, white diamond = mean (labelled). rollout+resample ranks pairs by sampled-completion vote frequency; pairwise by contact-statement log-prob; structure models by pyconfind degree on the predicted structure. Metrics computed by exp89's exact code.




```
$ plot_comparison.py --precision-csv _scratch/contact_precision_with_rollout.csv --out plots
```

cmp_contacts_at_R_by_fold_verdict.png

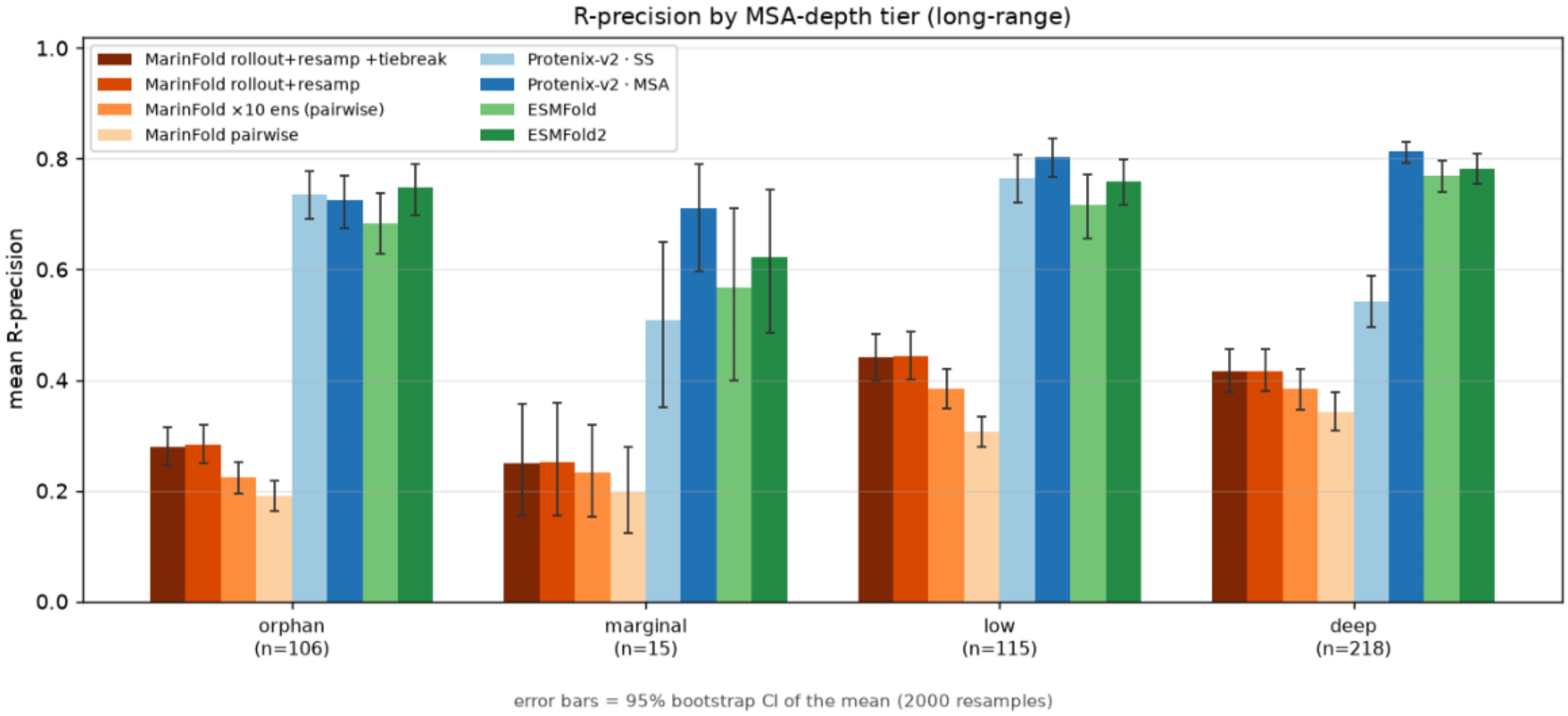
R-precision by fold novelty (long-range). Bars = mean; error bars = 95% bootstrap CI of the mean.




```
$ plot_comparison.py --precision-csv _scratch/contact_precision_with_rollout.csv --out plots
```

cmp_contacts_at_R_by_neff_tier.png

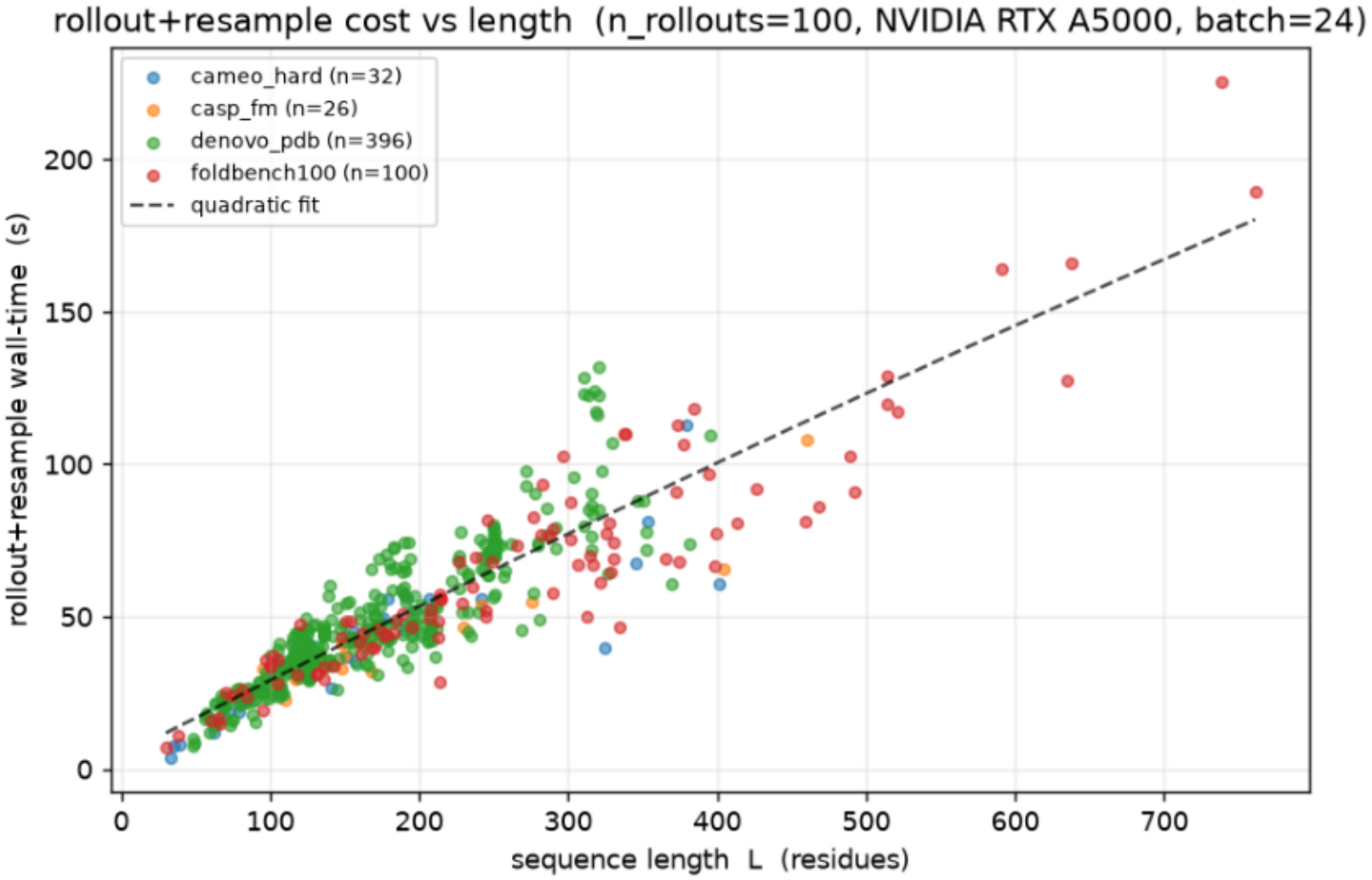
R-precision by MSA-depth tier (long-range). Bars = mean; error bars = 95% bootstrap CI of the mean.




```
$ plot_comparison.py --precision-csv _scratch/contact_precision_with_rollout.csv --out plots
```

eval_full_resample_time_vs_length.png

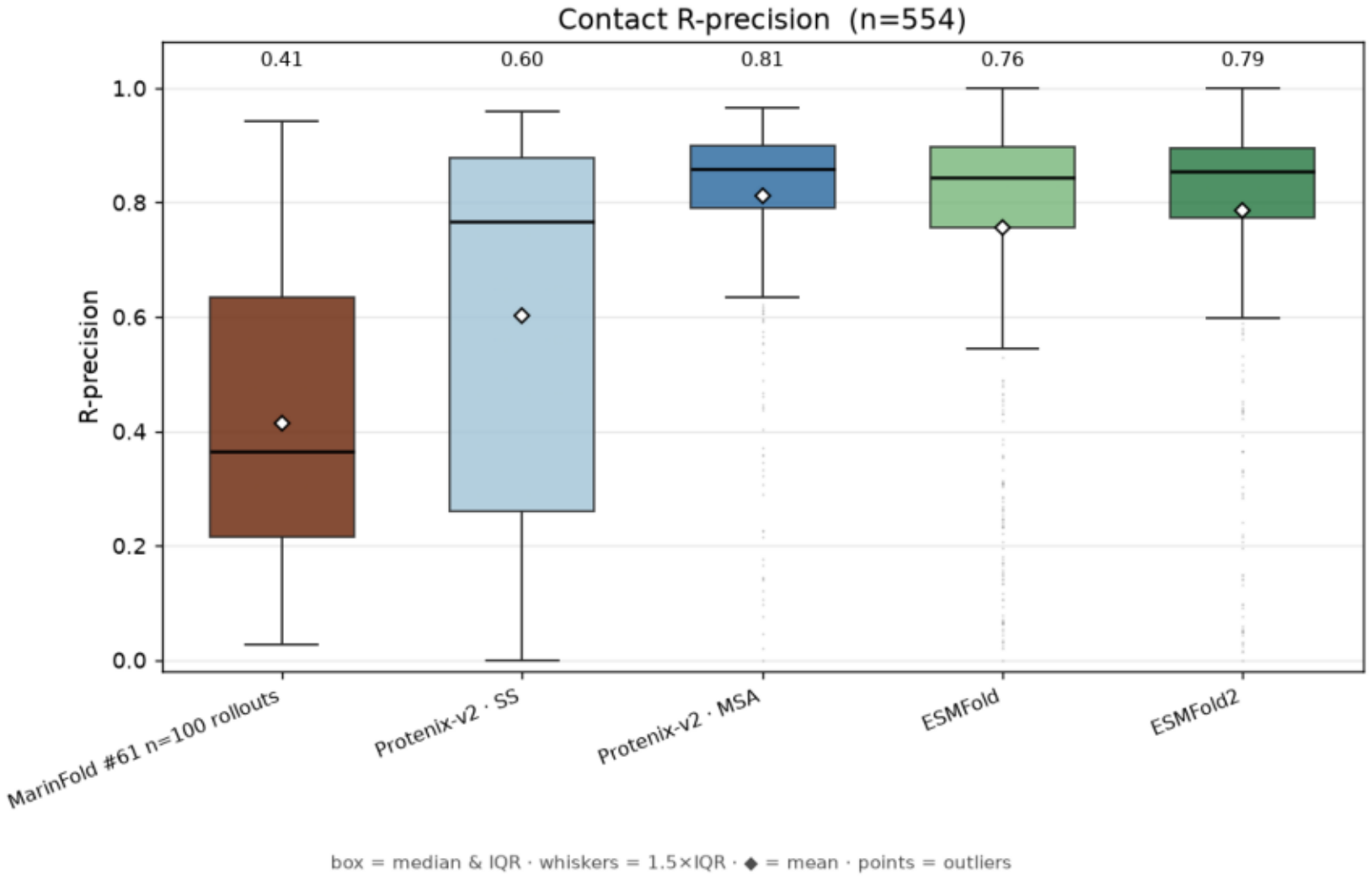
rollout+resample wall-time per protein vs sequence length (one point per protein, 554 curated eval proteins, n_rollouts=100, one A5000). Cost grows ~linearly with L (generation length) with mild super-linear curvature from attention.




```
$ analyze_full_curated_set.py --results _scratch/eval_full_results.jsonl
```

where_we_stand_rprecision.png

Contact R-precision (all sep>=6, n=554): MarinFold #61 n=100 rollouts (rollout+resample+tiebreak) vs Protenix-v2 / ESMFold / ESMFold2.




```
$ plot_where_we_stand.py --precision-csv _scratch/contact_precision_with_rollout.csv
```